Project 7 — 90-Day Remediation Roadmap

Author: Abhinav Prajapati · **Assessment date:** 2026-11-09 · **Day 0:** 2026-11-10

Three phases. Effort key: **Easy** = under 2 hours · **Medium** = under 1 day · **Hard** = more than 1 day. Every action has a responsible owner role and a tool/method. Target dates are relative to Day 0 (2026-11-10).

Phase 1 — Weeks 1–2 · Critical + easy wins · target by 2026-11-24

#	Action	Findings	Risk	Effort	Owner role	Tool / method
1.1	Remove secrets from the system prompt & config; **rotate** the exposed DB password and API key	F-03	Critical	Easy	AppSec Engineer	`os.environ.get()` + secret manager (Vault/AWS SM); revoke `sk-techcorp-9f3a1c7e2b`
1.2	Disable Flask debug; bind to localhost; use a production WSGI server	F-05	High	Easy	Backend Dev	`debug=False`, gunicorn/waitress behind reverse proxy
1.3	Add security headers and `HttpOnly`+`Secure` cookies	F-07	Medium	Easy	AppSec Engineer	X-Frame-Options, CSP, X-Content-Type-Options, HSTS at app/reverse-proxy
1.4	Put a path allow-list on MCP `read_file` (interim containment)	F-09	High	Easy	Backend Dev	`os.path.realpath(p).startswith(BASE_DIR)` else refuse

#	Action	Findings	Risk	Effort	Owner role	Tool / method
1.5	Disable Apache directory indexing; remove `/config`, `php.ini`, `phpinfo.php` exposure	F-07	Medium	Easy	Platform/ Ops	`Options -Indexes`; restrict web root

Phase 2 — Month 1 · High-risk items · target by 2026-12-10

#	Action	Findings	Risk	Effort	Owner role	Tool / method
2.1	Parameterise **all** SQL; migrate password hashing MD5 → bcrypt/argon2; add least-privilege DB account	F-01, F-02	Critical	Medium	Backend Dev	Prepared statements / ORM; `bcrypt`; scoped DB role
2.2	Remove or sandbox MCP `execute_command`; replace with a fixed allow-listed of argument-validated operations (no shell); require human approval	F-08	Critical	Hard	AppSec + Architect	No `shell=True`; capability allow-list; human-in-the-loop
2.3	Deploy AI input validation + output scanning on `/chat`	F-04, F-10	High	Medium	AppSec Engineer	Project 2 `validate.py` + `output_scanner.py`

#	Action	Findings	Risk	Effort	Owner role	Tool / method
2.4	Turn on the CI/CD security gates on every push	F-11	High	Medium	DevOps	Project 5 `security.yml` (Bandit, pip-audit/Safety, Gitleaks, `inject_scanner.py`)
2.5	Pin and patch all dependencies; break the build on known CVEs	F-12	High	Medium	DevOps	`requirements_pinned.txt`; pip-audit in CI gate 2

Phase 3 — Months 2–3 · Medium + architectural · target by 2027-02-10

#	Action	Findings	Risk	Effort	Owner role	Tool / method
3.1	Context-aware output encoding + Content-Security-Policy across the web app	F-06, F-07	Medium	Medium	AppSec Engineer	Template auto-escape; strict CSP
3.2	MCP capability redesign: least privilege per tool, per-tool authorization, defences against indirect/resource injection	F-08, F-09, F-10	High	Hard	Architect + AppSec	Scoped tool contracts; consent between chained calls

#	Action	Findings	Risk	Effort	Owner role	Tool / method
3.3	Monitoring, detection & logging for the AI and web layers	F-04, F-11	Medium	Medium	DevOps + SOC	SIEM / structured logging (NIST DE.CM, EU AI Act Art 12)
3.4	Regression-proofing: keep the gates mandatory and re-test all 12 findings before closure	all	—	Medium	AppSec Lead	Re-run Projects 1–6 checks as release criteria

Sequencing rationale: Phase 1 neutralises the already-exposed secrets and the cheapest Critical/High exposure first; Phase 2 fixes the high-impact structural flaws (SQLi, MCP RCE, AI validation) and installs the CI/CD gates so fixes cannot regress; Phase 3 closes the medium and architectural items and establishes the monitoring and re-test discipline that keeps TechCorp compliant over time.